

IncidentFlow: A Domain-Specific Language for Auditable Incident Response Playbooks

Abstract—We present IncidentFlow, a compact domain-specific language (DSL) and toolchain for authoring, validating and auditing incident-response playbooks. IncidentFlow targets operational teams and small-to-medium organizations that require precise, human-readable playbooks with automated validation and structured report generation. The system combines an ANTLR4 grammar, a Java-based compiler and a small HTTP API and interactive wizard to produce validated playbooks and concise incident reports. We describe the language design, compilation pipeline, CLI/API architecture, and reporting facilities, and provide a qualitative evaluation of usability, traceability, and limitations.

I. INTRODUCTION

Cybersecurity incident response operationalises the lifecycle prescribed by NIST SP 800-61 Rev. 3 [1] and ISO/IEC 27035 [2]: preparation, detection, containment, eradication, recovery, and post-incident learning. In organisational practice, the procedural component of this lifecycle is captured in playbooks — structured procedural documents that direct an analyst through a specific incident category. Despite their operational criticality, playbooks are typically maintained as static PDF, Word documents, or wiki pages. Analysts must consult them under acute time pressure while simultaneously triaging evidence and coordinating response activity.

The literature associates this practice with operational degradation: analysts spend substantial effort investigating false positives and sustained alert pressure correlates with step-skipping and documentation gaps [3]. Fewer than forty percent of organisations perform a formal post-incident review after every event [4]. Regulatory drivers (for example, GDPR Article 33 and the NIS2 Directive) increase the need for demonstrable, timestamped, attributable evidence that documented procedures were followed [5,6].

Commercial SOAR platforms (e.g., Splunk SOAR, Palo Alto XSOAR, IBM Resilient) encode playbooks as executable workflows; they provide rich integrations but impose vendor lock-in and operational cost that exclude many small and medium organisations. IncidentFlow targets the gap between informal static playbooks and heavyweight automation platforms. The project treats the playbook as a formal, parseable contract: a compact DSL that is both human-readable and machine-checked. From a single source artifact IncidentFlow produces validation diagnostics and a deterministic, auditable incident report.

This paper presents two contributions. First, a small lifecycle-aligned grammar implemented in ANTLR4 that captures common operational constructs (phases, triggers, verification, logging, waits, and explicit REPORT blocks). Second, a

compact engineering pipeline that converts the grammar into a usable command-line tool and HTTP API, together with an interactive wizard that accelerates creating validated playbooks for representative techniques.

II. RELATED WORK AND PROBLEM ANALYSIS

Incident response runbooks traditionally take the form of textual documents or wiki pages. Prior academic and industrial efforts have proposed structured playbook representations and automation platforms. SOAR systems (commercial) focus on action execution, integration with many third-party tools, and GUI-driven playbook composition. In contrast, incident documentation and human decision steps are poorly supported in such systems and can be costly for smaller teams.

From an engineering perspective, two problems motivate IncidentFlow. First, static playbooks lack validation: typographical errors, incorrect references, and inconsistent severity handling can produce ineffective responses at critical moments. Second, automation-centric platforms conflate orchestration with documentation, reducing human comprehensibility and audibility. IncidentFlow addresses these by separating specification (declarative procedural DSL) from execution: playbooks specify “what to do” and include explicit LOG, VERIFY, SEVERITY, and REPORT constructs so every decision is captured and reproducible in generated reports.

III. SYSTEM DESIGN AND IMPLEMENTATION

This section summarizes the core engineering decisions: language design, parsing and semantic checks, the CLI/API architecture, and reporting.

A. Language Design

The grammar is intentionally constrained to keep playbooks readable and amenable to static checks. A program consists of optional configuration blocks, optional imports, and one or more playbook declarations terminated by EOF. Each playbook declares a name, a mandatory ‘TRIGGER’ expression, one or more named ‘PHASE’s, and an optional ‘REPORT’ block. Phases contain statements drawn from a small set of primitives: ‘DO’ (action invocation), ‘LOG’, ‘VERIFY’ with ‘ON_FAIL’ handlers, branching(‘IF’/‘ELSE’), iteration(‘FOREACH’

This constrained vocabulary supports the project’s two primary goals: (1) make playbooks easy to author and review by humans, and (2) enable conservative static analyses that catch common operational errors (for example, undefined ‘GOTO’ targets, malformed ‘REPORT’ blocks, and missing ‘ON_FAIL’ handlers where ‘VERIFY’ is used).

For readers' convenience, the program skeleton is presented below:

```
CONFIG <name>:
  <key>: <value>
  ...

IMPORT "<path>" AS <alias>

PLAYBOOK <Name>
  TRIGGER: <expression>

  PHASE <name>:
    <statements>

  REPORT:
    <statements>

EOF
```

The full grammar is implemented in ANTLR4 (file: `grammar/IncidentFlow.g4`). The choice of ANTLR4 provides robust error reporting for syntax errors, a generated parse tree suitable for visitor-based semantic checks, and a straightforward code-generation path for the Java-based compiler.



Fig. 1. Parsing pipeline and example parse tree (ANTLR4).

The decision to implement the grammar in ANTLR4 yields several benefits: (1) robust parsing with informative syntax errors, (2) a generated parser available across platforms, and (3) a parse tree suitable for semantic visitors. The grammar (located at `grammar/IncidentFlow.g4`) deliberately keeps function-call and member-access syntax generic so the DSL can encode integration points without prescribing specific vendor APIs.

B. Compiler, Semantic Checking and CLI

The Java-based compiler is a small, focused pipeline that performs lexing, parsing, and a visitor-based semantic checker. The compiler's public API exposes `Compiler.fromSource(...)` and returns a `Result` object that carries the parse tree and explicit lists of syntax and semantic errors. Semantic checks are deliberately conservative: current checks ensure intra-playbook consistency (for example, every `GOTO` target refers to an existing phase) and basic configuration correctness. Error conditions are reported with line/column locations and returned to the CLI for deterministic exit codes, enabling integration in CI or pre-deployment checks.

The CLI, implemented in `Main.java`, routes commands to the compiler, wizard, report generator, or API server. Commands include `wizard`, `check`, `report`, `tree` (GUI parse-tree viewer), and `serve`. The CLI follows UNIX-friendly conventions: exit codes distinguish syntax errors, semantic

errors, and other failures, and `-o` allows writing the generated Markdown report to a file.

C. Interactive Wizard and HTTP API

An interactive `Wizard` guides operators through technique-specific questionnaires to synthesize playbooks for representative attack patterns (phishing T1566, PowerShell T1059.001, process injection T1055). The wizard programmatically constructs `.iflow` source text and then invokes the compiler and the `ReportVisitor` to produce a human-readable Markdown report. This pattern accelerates consistent playbook creation and captures operator decisions as part of the playbook source.

The small HTTP API mirrors CLI functionality: `/api/compile` validates source text and returns structured error lists; `/api/report` returns the generated Markdown report; file-backed endpoints permit serving stored playbooks. Responses are JSON with consistent error shapes and status codes suitable for embedding in CI pipelines or a simple web frontend.

D. Report Generation and Auditability

Report generation traverses the validated parse tree to produce a structured Markdown incident report. The `REPORT` block in the DSL provides explicit guidance on closure steps; in addition, per-phase `LOG` entries and `SEVERITY` changes are included in the generated output to create an audit trail. Generated reports contain: configuration metadata, a phase-by-phase statement breakdown, a severity timeline, and enumerated actions. Because reports are derived deterministically from the playbook source, the system provides a reproducible and human-verifiable audit trail without introducing side-effectful execution.

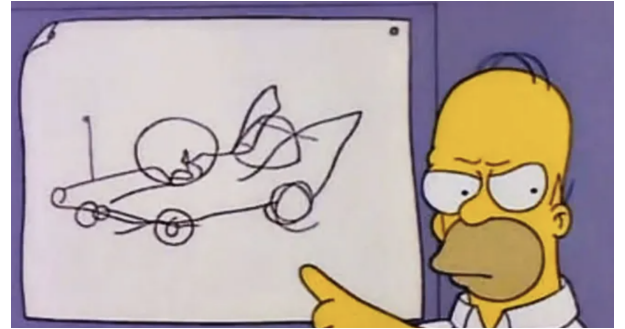


Fig. 2. Overall IncidentFlow processing architecture: playbook source → validation → report generation.

IV. EVALUATION AND CONCLUSION

We evaluate IncidentFlow along three dimensions: practical usability, traceability, and deployment suitability for small and medium organizations.

A. Practical Usability

IncidentFlow favors concise textual authoring over heavy GUI composition. The DSL's constructs map directly to common operational tasks (triage, containment, escalation).

The interactive wizard reduces the entry barrier by producing complete, validated playbooks for common techniques. The CLI and API permit automation (e.g., pre-commit checks) and integration with existing tooling.

B. Audit Traceability

By design, every decision that affects incident state is either a logged statement or a structured directive in the playbook (e.g., ‘VERIFY’, ‘SEVERITY’, ‘REPORT’). The deterministic report generator produces a reproducible artifact suitable for post-incident review and compliance reporting. Separating specification from execution ensures the report reflects the documented intent rather than external side effects.

C. Comparative Analysis and Limitations

Compared to enterprise SOAR platforms, IncidentFlow trades runtime orchestration and one-click connector breadth for simplicity, auditability, and low operational cost. It does not execute actions and therefore cannot replace automation where immediate enforcement is required. The semantic checking is currently conservative and can be extended: future work includes richer static analyses (data-flow checks, resource access modeling), formal verification of playbook termination (GOTO cycles), and a pluggable adapter layer to validate action argument schemas against real connectors.

D. Conclusion

IncidentFlow offers a lightweight, practical approach to specifying and validating incident response playbooks. By combining a compact DSL, ANTLR4-based parsing, lightweight semantic checks, and deterministic report generation, it improves the reliability and auditability of operational runbooks without increasing automation risk. The project is particularly suitable for small-to-medium teams that require structured, reproducible playbooks and straightforward integration into existing workflows.

REFERENCES